

به نام ایزد یکتا

برنامه سازی پیشرفته - 4041

استاد درس: آقای دکتر علی جاویدانی

اعضای تیم دستیاران دانشجویی: یارسا دولت آبادی - مبینا صفری - مهدیس یعقوب انصاری -

متین امیرپناه فر - شهاب ثمری شمس - محمد متین سلیمانی - نیما مخملی



تمرین اول | مروری بر CPP - دزد نمره

مهلت تحویل: 1404/08/02 - ساعت 23:59

مقدمه

در سال 1405، سیستم مدیریت دانشجوی دانشگاه بوعلی سینا با مشکل امنیتی جدی مواجه شده است. تعدادی از دانشجویان موفق شده‌اند نمرات و معدل خود را در سیستم قدیمی تغییر دهند و مدرک جعلی دریافت کنند. مسئولان دانشگاه تصمیم گرفتند سیستمی امن و مطمئن طراحی کنند که امکان تغییر یک پرونده تنها در صورتی وجود داشته باشد که تمام پرونده‌های بعدی نیز تغییر کنند؛ ایده‌ای که از فناوری بلاکچین الهام گرفته شده است.

در این سیستم، پرونده‌های دانشجویان به صورت زنجیره‌ای به هم متصل شده‌اند و هر پرونده شامل اطلاعات زیر است:

- نام دانشجو
- شماره دانشجویی
- تعداد واحدهای اخذ شده
- معدل
- کد امنیتی (که بر اساس اطلاعات پرونده و کد امنیتی پرونده قبلی محاسبه می‌شود)
- اشاره‌گری به پرونده دانشجوی بعدی

ساختار داده

برای نگهداری اطلاعات هر دانشجو از یک ساختار داده (struct) به نام Student استفاده کنید که به شکل زیر تعریف می‌شود:

```
struct Student {  
    string name;           // نام دانشجو  
    int studentID;         // شماره دانشجویی (شناسه یکتا)  
    int units;             // تعداد واحدهای اخذ شده  
    float gpa;             // معدل کل دانشجو  
    string securityCode;   // کد امنیتی پرونده  
    Student* next;         // اشاره گر به پرونده بعدی در زنجیره  
};
```

وظایف برنامه

برنامه باید قابلیت های زیر را داشته باشد:

1. افزودن پرونده جدید

- کاربر اطلاعات دانشجو را وارد می کند.
- هنگام افزودن پرونده جدید، کاربر علاوه بر اطلاعات دانشجو، **کد امنیتی دلخواه (مثلاً یک عدد ۴ رقمی)** را نیز وارد می کند.
- این کد امنیتی در پرونده ذخیره شده و در نمایش ها چاپ می شود.
- هنگام حذف یا بهروزرسانی پرونده ها، کد امنیتی دست نخورده باقی می ماند (تغییری در آن ایجاد نمی شود).
- پرونده جدید به انتهای زنجیره اضافه می شود.

2. حذف پرونده بر اساس شماره دانشجویی

- اگر دانشجو وجود داشت، حذف شود و زنجیره بدون مشکل ادامه پیدا کند.

3. جستجوی دانشجو بر اساس شماره دانشجویی

- اگر پیدا شد، تمام اطلاعاتش نمایش داده شود.

4. بهروزرسانی نمرات دانشجو

- نمرات قبلی آزاد شود (حافظه پویا).
- نمرات جدید جایگزین شود.
- کد امنیتی این پرونده و همه پرونده های بعدی دوباره محاسبه شود.

5. نمایش همه پرونده ها

- از اولین تا آخرین، اطلاعات کامل و کد امنیتی چاپ شود.

توابع

```
void addStudent(Student*& head, const string& name, int studentID, int units, float gpa, const string& securityCode);
```

وظیفه:

- ایجاد یک پرونده جدید با اطلاعات ورودی: نام، شماره دانشجویی، تعداد واحدها، معدل و کد امنیتی.
- تخصیص حافظه پویا برای این پرونده (با new).
- افزودن این پرونده به انتهای زنجیره لیست پیوندی.
- اگر لیست خالی بود، این پرونده به عنوان اولین عنصر قرار می‌گیرد.
- اشاره‌گر next پرونده جدید باید مقدار nullptr باشد.

```
bool deleteStudent(Student*& head, int studentID);
```

وظیفه:

- جستجو در لیست برای پیدا کردن پرونده‌ای که شماره دانشجویی آن برابر studentID است.
- اگر پرونده پیدا شد، آن را حذف کرده و حافظه مربوطه را آزاد می‌کند (delete).
- اصلاح اشاره‌گرهای لیست به گونه‌ای که زنجیره همچنان پیوسته و صحیح باقی بماند.
- اگر پرونده حذف شود، مقدار بازگشتی true و اگر پیدا نشود false برگردانده شود.
- اگر پرونده مورد حذف اولین گره لیست باشد، رأس لیست باید به گره بعدی تغییر کند.

```
Student* searchStudent(Student* head, int studentID);
```

وظیفه:

- پیمایش لیست پیوندی از ابتدای آن به دنبال پرونده‌ای با شماره دانشجویی studentID بگردد.
- اگر پیدا شد، اشاره‌گر به آن پرونده را برگرداند.
- اگر پیدا نشد، مقدار nullptr بازگرداند.
- این تابع اطلاعات پرونده را فقط جستجو می‌کند و تغییری نمی‌دهد.

```
bool updateStudent(Student* head, int studentID, int newUnits, float newGPA);
```

وظیفه:

- جستجوی پرونده با شماره دانشجویی studentID.
- اگر پیدا شد، تعداد واحدها و معدل آن را به مقادیر جدید newUnits و newGPA بهروزرسانی کند.
- در این تمرین نیازی به تغییر کد امنیتی نیست (در نسخه ساده).
- مقدار بازگشتی true اگر بهروزرسانی موفق باشد و false اگر پرونده پیدا نشود.

```
void displayAllRecords(Student* head);
```

وظیفه:

- پیمایش کل لیست از ابتدا تا انتها.
- چاپ اطلاعات کامل هر پرونده شامل: نام، شماره دانشجویی، تعداد واحد، معدل و کد امنیتی.
- چاپ هر پرونده در خط جداگانه یا قالبی خوانا.
- اگر لیست خالی باشد، پیام مناسبی مثل "لیست پرونده‌ها خالی است" نمایش داده شود.

نکته کلی در همه توابع:

- حتماً در کار با اشارهگرها دقت شود تا دسترسی به حافظه غیرمجاز (dangling pointer) ایجاد نشود.
- حافظه‌ای که آزاد می‌شود (در حذف) باید با delete انجام شود و بعد از آن اشارهگر مربوطه باید اصلاح شود.
- در افزودن و حذف اگر لیست خالی یا تک عنصری باشد، شرایط ویژه باید مدیریت شود.

محدودیت‌ها

- استفاده از `vector`, `list` یا هر ساختار آماده کتابخانه‌ای ممنوع است.
- استفاده از فایل (`fstream`) ممنوع است.
- ظرفیت باید نامحدود باشد (مدیریت حافظه پویا).
- مدیریت حذف، اضافه و تغییر داده باید با اشاره‌گرها انجام شود.

بارمندی تمرین (۱۰۰ نمره):

- تعریف ساختار داده و مدیریت اشاره‌گرها : ۱۰ نمره
- افزودن پرونده جدید با حافظه پویا : ۲۰ نمره
- حذف پرونده و اصلاح زنجیره : ۲۰ نمره
- جستجو بر اساس شماره دانشجویی : ۱۰ نمره
- بهروزرسانی نمرات : ۱۰ نمره
- نمایش کامل پرونده‌ها : ۱۰ نمره
- رعایت محدودیت‌ها : ۱۰ نمره
- پاکیزگی کد و خوانایی : ۱۰ نمره

نحوه تحویل 📁

- تمامی فایل‌های مرتبط با تمرین، شامل سورس‌کد و هرگونه مستندات یا توضیحات تکمیلی، باید در یک پوشه با نام زیر قرار گیرند:

HW1-StudentNumber

- این پوشه را فشرده کرده و با فرمت `zip` ارسال نمایید.
- ارسال فایل تنها از طریق بخش "تمرین اول" در کلاس درس، در سامانه کونرا صورت می‌گیرد.

لطفاً پیش از اتمام مهلت مقرر، از آپلود صحیح فایل اطمینان حاصل فرمایید.

گفتم دل رحیمت کی عزم صلح دارد؟ گفتا مگوی با کس تا وقت آن درآید...